Lab 05
Data Structures
BS DS Fall 2025 Morning/Afternoon

Task 1 **20 marks**

Write a function that solves a maze problem. In this particular version of the problem, a hiker must find a path to the top of a mountain. Assume that the hiker leaves a parking lot, marked **P**, and explores the maze until he reaches the top of a mountain, marked **T**.



The maze is represented as a 2D array of characters. The character `*` represents a barrier, while `P` and `T` represent the parking lot and mountain top, respectively. A blank space represents an open cell through which a path can be taken.

Write a **recursive function** that attempts to find a path from `P` (parking lot) to `T` (mountain top). The function should return **"Solved"** if a path exists and **"UnSolved"** if no path can be found. If the maze is solved, the function should also print the complete path from `P` to `T`. The path may be printed in **reverse order**.

You have to design a **solve_maze** function.

```
bool solve_maze(char maze[6][6], Pair s, Pair d, int row, int col)
```

Test your function using the following driver program

```
int main()
{
    // -------------------------
    // Maze 1
    // -------------------------

    char maze1[6][6] = {
        {' ', '*', ' ', '*', ' ', ' '},
        {' ', '*', ' ', '*', ' ', ' '},
        {'P', ' ', ' ', ' ', '*', ' '},
        {'*', ' ', '*', '*', '*', ' '},
        {' ', ' ', ' ', ' ', '*', 'T'},
        {'*', ' ', ' ', ' ', ' ', ' '}
    };

    Pair start = {2, 0};
    Pair dest  = {4, 5};
       cout<<"Maze 1"<<endl;
    if (solve_maze(maze1, start, dest, 6, 6))
    {
        cout << "  Solved" << endl;
    }
    else
    {
        cout << "UnSolved" << endl;
    }


    // -------------------------
    // Maze 2
    // -------------------------

    char maze2[6][6] = {
        {' ', '*', ' ', '*', ' ', ' '},
        {' ', '*', ' ', '*', ' ', ' '},
        {'P', ' ', ' ', ' ', '*', ' '},
        {'*', '*', '*', '*', '*', ' '},
        {' ', ' ', ' ', ' ', '*', 'T'},
        {'*', ' ', ' ', ' ', ' ', ' '}
    };
       cout<<"Maze 2"<<endl;
    if (solve_maze(maze2, start, dest, 6, 6))
    {
        cout << "  Solved" << endl;
    }
    else
    {
        cout << "UnSolved" << endl;
    }

    return 0;
}
```

The output of the following program should be

```
Maze 1
(4,5) (5,5) (5,4) (5,3) (4,3) (4,2) (4,1) (3,1) (2,1) (2,0)   Solved
Maze 2
UnSolved
```

Task 2 — 20 marks

Write the recursive function reverse(n) to return the integer obtained by reversing the digits of its integer parameter. Test your function using the following driver.

```
int main()
{
   int n=56321;
   cout << "Reversed number: " << reverse(n) << endl;

         n=89641;
   cout << "Reversed number: " << reverse(n) << endl;
   return 0;
}
```

The output of the following program should be

```
Reversed number: 12365
Reversed number: 14698
```

Task 3 — 20 marks

A password is considered valid if characters at even indices are digits and characters at odd indices are lowercase letters.

Sr #	Input	Expected Output	Description
1	1a2b3c	Valid	Even indices contain digits and odd indices contain lowercase letters.
2	1a2B3c	Invalid	B at index 3 is not a lowercase letter.
3	12ab34	Invalid	2 at index 1 should be a lowercase letter.

Write a recursive function that takes a password string and determines whether it is Valid or Invalid.
bool isValidPassword(const string &s, int index = 0);
Use the following drive to test your function.

```
int main() {
  string passwords[] = {"1a2b3c","23478","1a2B3c","5x7y"   };
   for (string password : passwords)
   {
      bool is_valid = isValidPassword(password, 0);
      cout << "Password: " << password
          << " is "
          << (is_valid ? "valid" : "invalid")
          << endl;
   }
   return 0;
}
```

The output of the following program should be

```
Password: 1a2b3c is valid
Password: 23478 is invalid
Password: 1a2B3c is invalid
Password: 5x7y is valid
```

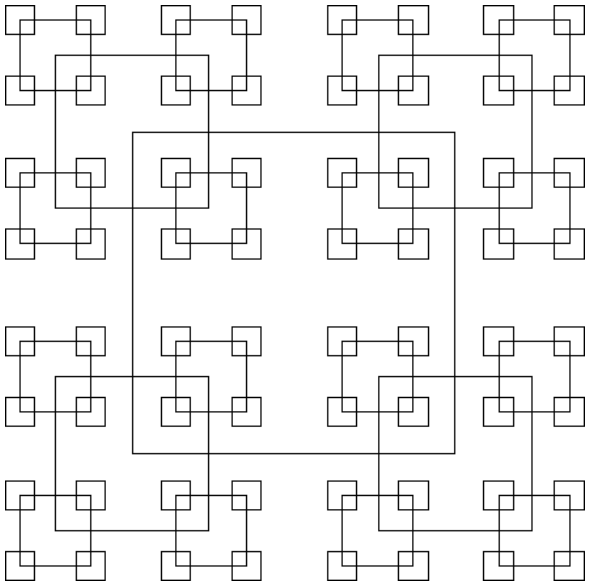
Task 4 — 40 marks

Geometrically, any square has a unique, well-defined centre point. On a grid this is only true if the sides of the square are an odd number of points long. Since any odd number can be written in the form 2k+1, we can characterize any such square by specifying k, that is we can say that a square whose sides are of length 2k+1 has size k. Now define a pattern of squares as follows.

1. The largest square is of size k (that is sides are of length 2k+1) and is centred in a grid of size 1024 (that is the grid sides are of length 2049).
2. The smallest permissible square is of size 1 and the largest is of size 512, thus $1 \leq 512 \leq k$.
3. All squares of size $k > 1$ have a square of size ***k div 2*** centred on each of their 4 corners. (Div implies integer division, thus 9 div 2 = 4).
4. The top left corner of the screen has coordinates (0,0), the bottom right has coordinates (2048, 2048).

Hence, given a value of k, we can draw a unique pattern of squares according to the above rules.

Furthermore any point on the screen will be surrounded by zero or more squares. (If the point is on the border of a square, it is considered to be surrounded by that square). Thus if the size of the largest square is given as 15, then the following pattern would be produced.



Write a program that will read in a value of k and the coordinates of a point, and will determine how many squares surround the point.

Input and Output

Input will consist of three integers, first integer shows the value of k and other two integers shows the coordinates of a point.

Output will consist of a integer which shows the number of squares containing the specified point.

Sample input 1

500 113 941

Sample output 1

5

Sample input 2

300 100 200

Sample output 2

0

Sample input 3

300 1024 1024

Sample output 3

1